

AI-Powered Project Management Platform

Multi-model AI integration with GitHub-driven configuration, employee-specific access control, and extensible skill management.

DOCUMENT TYPE

Technical Architecture & Implementation Plan

TECH STACK

Next.js 16 / Node.js / PostgreSQL / GitHub API

AI MODELS

GLM (Z.ai) / OpenAI / Anthropic / Open-Source

DATE

July 2026

Table of Contents

1. Executive Summary and Vision	2
2. System Architecture Overview	2
2.1 Component Breakdown	2
2.2 Architecture Diagram	3
3. Multi-Model AI Integration Layer	4
3.1 Model Comparison and Use Cases	4
3.2 Adapter Pattern and Model Router	4
4. Security and Access Control	5
4.1 Authentication Flow	5
4.2 Role-Based Access and Session Isolation	6
5. GitHub-Driven Configuration System	6
5.1 Repository Structure	6
5.2 Webhook-Driven Sync and Deployment	7
6. Skill Sets and Shortcut Commands	8
6.1 Slash Command System	8
7. Enhanced Features and Innovation Ideas	9
7.1 AI Meeting Summaries and Action Item Extraction	9
7.2 Automated Task Generation from Conversations	9
7.3 Project Risk Prediction Engine	9
7.4 Cross-Project Knowledge Graph	9
7.5 Admin Analytics Dashboard	10
8. Implementation Roadmap	10
8.1 Technology Stack Summary	11
8.2 Team Composition	11
8.3 Risk Mitigation	11

1. Executive Summary and Vision

This document presents a comprehensive architecture and implementation blueprint for integrating large language models (LLMs) into an existing project management platform. The system enables organizations to provide controlled, secure, and context-aware AI assistance to every employee through a unified chat interface embedded directly within the project management workflow. Rather than treating AI as an external tool, this design weaves intelligence into the fabric of daily project operations, from task decomposition and risk analysis to meeting summaries and cross-project knowledge retrieval.

The core innovation lies in three interconnected pillars. First, a **multi-model orchestration layer** that abstracts away provider-specific APIs behind a unified adapter pattern, allowing seamless switching between GLM (Z.ai), OpenAI GPT, Anthropic Claude, and self-hosted open-source models based on cost, capability, and data-privacy requirements. Second, a **GitHub-driven configuration system** where all AI behavior protocols, skill definitions, and model parameters live in a version-controlled repository, enabling audit trails, peer review via pull requests, and zero-downtime configuration updates through webhooks. Third, a **security-first access model** that ties every AI interaction to an authenticated employee identity, enforces per-session security codes, and maintains strict conversational isolation between team members.

Beyond these foundational pillars, the blueprint explores a rich set of value-added features: slash-command skill registries that let administrators define domain-specific AI workflows; automated task generation from chat conversations; real-time streaming for responsive interactions; vector-based knowledge graphs for project-specific context retrieval; and comprehensive admin analytics dashboards for monitoring usage patterns, cost allocation, and compliance. The target audience for this document is the engineering team responsible for building and deploying the platform, with sufficient detail to guide architecture decisions while remaining accessible to technical stakeholders evaluating the strategic direction.

2. System Architecture Overview

The platform follows a layered architecture that cleanly separates concerns across four tiers: a Presentation Layer handling user-facing interfaces, an API and Orchestration Layer managing request routing and AI model coordination, a Service Layer encapsulating business logic for authentication, configuration, and session management, and a Data Layer providing persistent storage, caching, and vector search capabilities. This separation ensures that each tier can evolve independently, whether swapping the frontend framework, adding a new AI provider, or migrating the database engine.

2.1 Component Breakdown

The **Next.js Frontend** serves as the primary web interface, built with React Server Components and Tailwind CSS for a responsive, performant user experience. Alongside it, an **Admin Dashboard** provides a separate interface for configuring AI skills, managing employee access policies, and monitoring system health. The **Employee Portal** is the chat-centric interface where staff interact with AI models, accessible only after multi-factor authentication including a unique security code. A **React Native Mobile App** extends the platform to mobile devices, sharing the same API layer and authentication system.

At the orchestration level, the **API Gateway** handles authentication verification, rate limiting, and intelligent request routing. The **AI Orchestrator** is the central brain: it selects the optimal model for each request based on task type, user permissions, and cost constraints, then manages the conversation context window, applies protocol-defined system prompts from the GitHub repository, and streams responses back through a **WebSocket Server** for real-time delivery. The service layer includes dedicated microservices for authentication (JWT tokens plus security-code verification), GitHub synchronization (webhook listeners and config parsing), a skill engine (slash-command resolution and dynamic prompt composition), and a session manager (per-employee chat isolation with complete audit trails).

2.2 Architecture Diagram

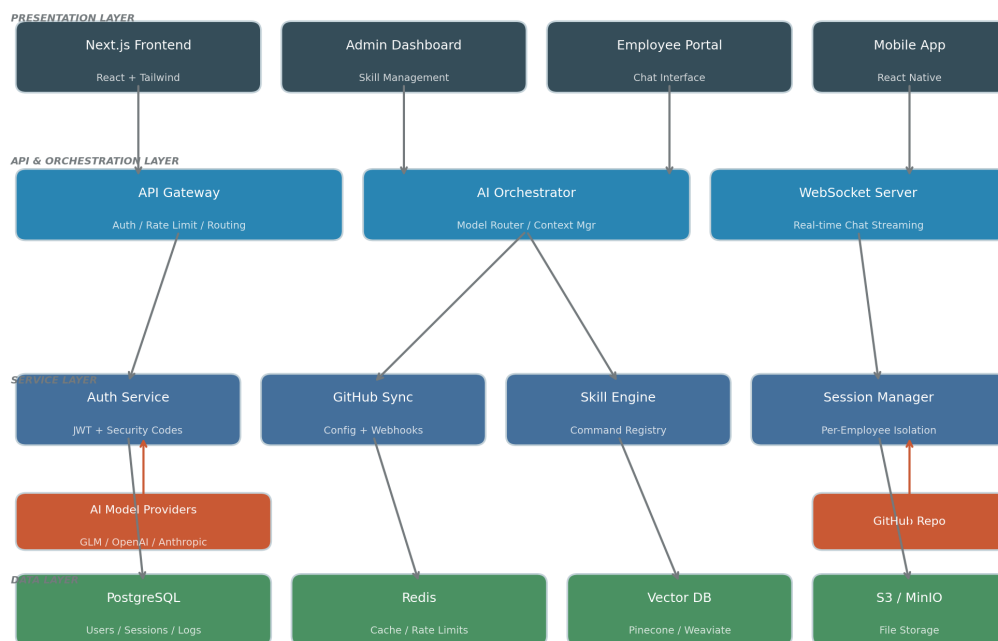


Figure 1. High-level system architecture showing the four-tier layout, external integrations with GitHub and AI providers, and internal service communication paths.

3. Multi-Model AI Integration Layer

A critical design goal is **model agnosticism**: the platform must never be locked into a single AI provider. The integration layer achieves this through an adapter pattern where each provider (GLM, OpenAI, Anthropic, or a locally-hosted model) implements a uniform interface contract. The AI Orchestrator holds references to all registered adapters and selects one at runtime based on a routing policy defined in the GitHub configuration repository. This means adding a new model requires only implementing the adapter interface and updating a YAML configuration file; no changes to the core application code are necessary.

3.1 Model Comparison and Use Cases

Provider	Models	Strengths	Best Use Case	Cost Tier
GLM (Z.ai)	GLM-4, GLM-4-Plus	Strong multilingual support, competitive pricing	General chat, code assistance, Chinese content	Low-Medium
OpenAI	GPT-4o, GPT-4o-mini	Broad capability, large ecosystem	Complex reasoning, function calling	Medium-High
Anthropic	Claude 4, Claude Haiku	Long context (200K tokens), safety	Document analysis, compliance review	Medium-High
Open-Source	Llama 4, Mistral, Qwen	Full data control, no API costs	Sensitive data, offline, custom fine-tuning	Low (infrastructure)

Table 1. AI model provider comparison for integration planning.

3.2 Adapter Pattern and Model Router

Each adapter implements a **ModelAdapter** interface with standardized methods: **complete()** for synchronous responses, **stream()** for token-by-token streaming, **embed()** for generating vector embeddings, and **countTokens()** for usage metering. The Model Router evaluates several signals before selecting an adapter: the task type tag from the slash command (e.g., `/code` routes to GPT-4o or GLM-4), the user role permissions table (executives may access premium models while junior staff use cost-efficient ones), the current monthly budget consumption per project, and a configurable fallback chain if the primary model is unavailable or returns an error. The router configuration itself lives in the GitHub repository under `config/model-routing.yaml`, making routing policies version-controlled and reviewable through standard pull-request workflows.

Streaming is implemented through Server-Sent Events (SSE) over the WebSocket connection, ensuring that employees see AI responses as they are generated rather than waiting for the complete output. This is particularly important for long-running tasks like project plan generation or risk analysis reports. The

orchestrator also maintains a sliding context window that summarizes older conversation turns to stay within token limits while preserving the most relevant context for follow-up questions. Conversation summaries are generated automatically when the context exceeds a configurable threshold, stored in the session database, and injected as system-prompt context for subsequent interactions.

4. Security and Access Control

Security is the foundational constraint of the entire system. Every AI interaction must be traceable to a specific authenticated employee, every conversation must be cryptographically isolated from other users, and every configuration change must go through an auditable approval process. The security model operates on three principles: **verify first** (never trust a request without multiple authentication factors), **isolate always** (each employee sees only their own chat history and never accesses another user's sessions), and **audit everything** (every prompt, response, and configuration change is logged with timestamps, user IDs, and IP addresses).

4.1 Authentication Flow

When an employee attempts to access the AI chat feature, the system enforces a multi-step authentication process. First, the employee must be logged into the project management platform with valid credentials (email/password plus optional SSO integration). Second, when they click "Open AI Chat," the system generates a time-limited, one-time **security code** (a 6-digit alphanumeric token) that is delivered via a secondary channel: either the employee's registered email or an authenticator app push notification. This security code is unique per session attempt and expires after 5 minutes. Third, the employee enters the code, and upon successful verification, the system creates an isolated chat session bound to their employee ID and role. The session token is a signed JWT that includes the employee ID, role, permissions scope, and the session creation timestamp.

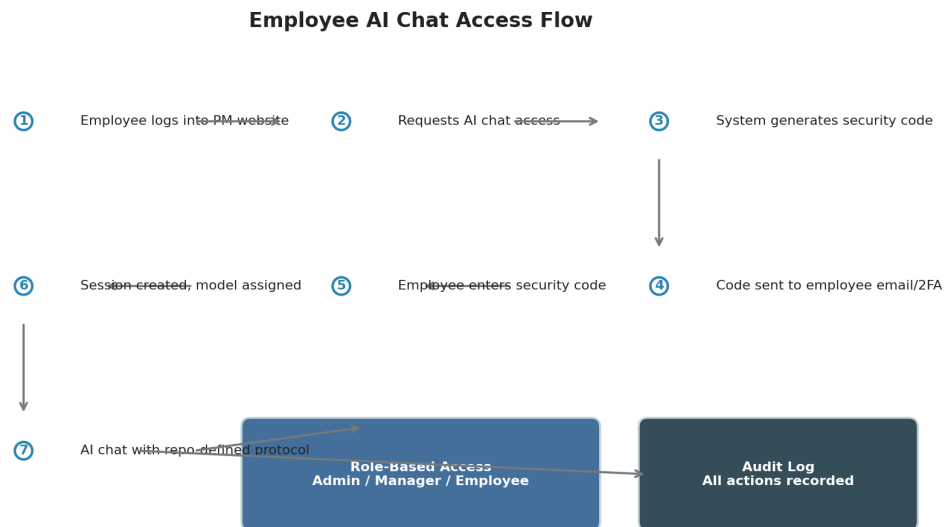


Figure 2. Employee AI chat access flow showing the security-code verification sequence and role-based access enforcement.

4.2 Role-Based Access and Session Isolation

The platform defines three primary roles with distinct AI access levels. **Administrators** have full control: they can configure which models are available, define skill sets and slash commands, view all employee usage analytics, and manage security policies. **Project Managers** can access AI within their assigned projects, view team-level (but not individual) usage summaries, and create project-specific skill configurations that override global defaults. **Employees** can only interact with AI within their personal sessions, use the skills and commands assigned to their role, and view their own chat history. No employee can access another employee's chat sessions, even if they share the same project or team. Session data is encrypted at rest using AES-256 and in transit via TLS 1.3, with encryption keys managed through a dedicated key vault service (HashiCorp Vault or AWS KMS).

5. GitHub-Driven Configuration System

One of the most distinctive features of this architecture is the decision to store all AI configuration in a GitHub repository rather than a traditional database or admin panel. This approach leverages Git's version control, branching, and pull-request workflow to create a transparent, auditable, and collaborative configuration management process. Every change to AI behavior goes through code review, every deployment is tied to a specific commit hash, and rollbacks are as simple as reverting a commit. The repository serves as the single source of truth for how AI models should behave, what skills are available, and which protocols govern each interaction type.

5.1 Repository Structure

Path	Purpose	Format
config/model-routing.yaml	Defines which model handles which task type, fallback chains, and per-role model access policies	YAML
config/protocols/	System prompts, behavior rules, and guardrails for each AI use case (e.g., code-review.yaml, risk-analysis.yaml)	YAML
skills/	Skill definitions: name, trigger command, system prompt template, required model capabilities, and parameter schema	YAML + Markdown
skills/templates/	Prompt templates with variable placeholders that the skill engine populates at runtime	Markdown
permissions/	Per-role and per-project access control lists defining which skills, models, and data each employee role can access	YAML
webhooks/	Configuration for outbound webhooks that trigger when specific AI events occur (e.g., task-created, risk-flagged)	JSON
docs/	Human-readable documentation for administrators explaining the configuration schema and change process	Markdown

Table 2. GitHub repository structure for AI configuration.

5.2 Webhook-Driven Sync and Deployment

A **GitHub Sync Service** runs as a background worker that listens for repository events through GitHub webhooks. When a push is made to the main branch, the service pulls the latest configuration, validates the YAML schemas against a defined JSON Schema, and hot-reloads the running AI orchestrator without requiring a server restart. For changes on feature branches, the system runs configuration validation in a dry-run mode and posts the results as a check on the pull request, giving reviewers immediate feedback on whether a proposed configuration change is valid. This creates a development-like workflow for AI configuration: administrators propose changes via branches, reviewers validate behavior through PR comments and automated checks, and the merge to main triggers an atomic configuration update. The sync service also maintains a local cache of the repository state so that the AI orchestrator can read configuration files without network latency, and it logs every configuration version with its commit hash for full audit traceability.

6. Skill Sets and Shortcut Commands

Skills are reusable, named AI workflows that employees invoke through slash commands or keyboard shortcuts. Each skill encapsulates a specific capability: generating a project status report, analyzing task dependencies for bottlenecks, drafting a risk mitigation plan, or summarizing a meeting transcript. Skills are defined as YAML files in the GitHub repository and loaded dynamically by the Skill Engine at runtime, meaning new skills can be added without deploying application code. Administrators assign skills to roles and projects through the permissions configuration, giving fine-grained control over which employee sees which commands in their chat interface.

6.1 Slash Command System

When an employee types a slash command (e.g., `/risks` or `/standup`), the Skill Engine resolves the command to a skill definition, loads the associated prompt template, populates it with context variables (project name, team members, recent task data fetched from the PM database), and sends the composed prompt to the AI model selected by the router. The template system supports conditional sections, loops, and variable interpolation, enabling sophisticated prompt engineering without hardcoding logic in the application. Employees can also create personal keyboard shortcuts that map to frequently used slash commands, providing quick access to common workflows.

Command	Skill Name	Description	Model Preference
<code>/standup</code>	Daily Standup Generator	Generates a concise standup summary from recent task updates and commits	Fast model (mini)
<code>/risks</code>	Risk Analyzer	Identifies potential project risks based on task delays, dependencies, and team workload	Reasoning model
<code>/summarize</code>	Meeting Summarizer	Summarizes meeting transcripts into action items, decisions, and open questions	Long-context model
<code>/code-review</code>	Code Review Assistant	Reviews code diffs for bugs, style issues, and security vulnerabilities	Code-specialized model
<code>/plan</code>	Task Decomposer	Breaks a high-level goal into granular, assignable tasks with estimates	Reasoning model
<code>/knowledge</code>	Project Knowledge Search	Searches across project documents, past decisions, and chat history using vector embeddings	Embedding + LLM

Table 3. Example skill set with slash commands and model routing preferences.

7. Enhanced Features and Innovation Ideas

Beyond the core architecture, several high-value features can significantly amplify the platform's impact on team productivity and project outcomes. These features are designed to be implemented in iterative phases, with each building on the infrastructure established by the foundational components described in earlier chapters.

7.1 AI Meeting Summaries and Action Item Extraction

Integrate with calendar and video-conferencing APIs (Google Meet, Zoom, Microsoft Teams) to automatically capture meeting transcripts. The AI processes these transcripts to extract action items with assigned owners and deadlines, key decisions made, and open questions requiring follow-up. Extracted action items are pushed directly into the project management task board, eliminating the manual transcription step that often delays task creation by hours or days. The system can also detect recurring meeting patterns and proactively suggest agenda items based on unresolved items from previous sessions.

7.2 Automated Task Generation from Conversations

When an employee discusses a new feature, bug fix, or project milestone with the AI, the system can automatically detect actionable statements and offer to create corresponding tasks in the project management system. For example, if a developer types "We need to refactor the authentication module before the sprint ends," the AI identifies this as a potential task and presents a pre-filled task card with a title, description, estimated effort, and suggested assignee based on team expertise profiles. The employee confirms or edits the card before it is created, ensuring human oversight while dramatically reducing the friction of task creation.

7.3 Project Risk Prediction Engine

By analyzing historical project data (task completion rates, scope changes, team velocity trends, and external dependency timelines), the AI can compute a dynamic risk score for each active project. Risk factors are categorized into schedule risk, resource risk, scope creep risk, and technical debt risk, each with specific leading indicators. When a risk score crosses a configurable threshold, the system generates an alert to project managers with a natural-language explanation of the risk, its likely impact, and recommended mitigation strategies drawn from historical project outcomes. This transforms the AI from a reactive chat tool into a proactive project intelligence system.

7.4 Cross-Project Knowledge Graph

Using vector embeddings generated by the AI models, the platform builds a searchable knowledge graph that spans all projects, documents, code repositories, and chat histories. When an employee asks a question, the system first performs semantic search across this knowledge graph to retrieve relevant context, then composes a response that references specific past decisions, similar issues resolved in other

projects, or relevant documentation. This capability is particularly valuable for onboarding new team members, avoiding duplicated work across projects, and preserving institutional knowledge that would otherwise exist only in individual developers' memories.

7.5 Admin Analytics Dashboard

A comprehensive analytics dashboard gives administrators real-time visibility into AI usage across the organization. Key metrics include total token consumption per project, cost breakdown by model and task type, most-used skills and slash commands, average response latency, employee engagement scores, and quality feedback ratings submitted by users after each interaction. The dashboard supports drill-down from organization-level aggregates to individual employee usage patterns, enabling data-driven decisions about model procurement, skill development priorities, and budget allocation. Export to CSV and scheduled email reports ensure that stakeholders without direct dashboard access receive regular updates.

8. Implementation Roadmap

The implementation follows a four-phase approach designed to deliver value incrementally while building the technical foundation required for more advanced features. Each phase produces a shippable increment that can be deployed to production and used by real employees, ensuring continuous feedback loops that inform subsequent development.

Phase	Duration	Deliverables	Key Milestones
Phase 1: Foundation	Weeks 1-4	Next.js frontend with chat UI, auth service with security codes, PostgreSQL schema, basic API gateway	Employee login + security-code flow working; single chat session with one AI model (GLM)
Phase 2: Core AI	Weeks 5-8	Multi-model adapter layer, AI orchestrator with routing, WebSocket streaming, session isolation, GitHub sync service	Three models operational (GLM, OpenAI, Anthropic); config changes via GitHub PRs deploy automatically
Phase 3: Advanced Features	Weeks 9-14	Skill engine with slash commands, meeting summarizer, task auto-creation, vector knowledge graph, admin dashboard	Six slash commands live; meeting-to-task pipeline working; knowledge search returning relevant results
Phase 4: Scale and Optimize	Weeks 15-20	Mobile app, risk prediction engine, multi-tenant support, cost optimization, advanced analytics, SSO integration	Mobile app in app stores; risk scores computed daily for all projects; cost per interaction reduced by 30%

Table 4. Four-phase implementation roadmap with durations, deliverables, and milestones.

8.1 Technology Stack Summary

The primary technology choices reflect a balance of developer productivity, ecosystem maturity, and long-term maintainability. The **frontend** is built with Next.js 16 (App Router), React 19, and Tailwind CSS 4, providing server-side rendering, streaming support, and a utility-first styling approach that accelerates UI development. The **backend** runs on Node.js with Express or Fastify, chosen for its non-blocking I/O model that handles concurrent WebSocket connections and streaming AI responses efficiently. **PostgreSQL** serves as the primary database for relational data (users, sessions, projects, audit logs), while **Redis** handles caching, rate limiting, and session token storage. **Pinecone** or **Weaviate** provides the vector database for semantic search and knowledge graph embeddings. **Docker** and **Kubernetes** manage containerized deployment, with **GitHub Actions** driving CI/CD pipelines. Infrastructure is provisioned through **Terraform** on AWS or GCP, with monitoring via **Grafana** and **Prometheus**.

8.2 Team Composition

A cross-functional team of five to seven engineers is recommended for the initial build. One full-stack developer focuses on the Next.js frontend and React Native mobile app. Two backend engineers handle the API layer, AI orchestrator, and service integrations. One DevOps engineer manages infrastructure, CI/CD, and monitoring. One AI/ML engineer designs the model adapter layer, prompt engineering workflows, and vector search system. A part-time technical writer maintains the GitHub configuration documentation and admin guides. For phases 3 and 4, the team can expand to include a dedicated data engineer for the analytics pipeline and a QA engineer for end-to-end testing of the security and access-control flows.

8.3 Risk Mitigation

Risk	Impact	Mitigation Strategy
AI provider API downtime	High - all chat functionality stops	Multi-model fallback chain with automatic failover; local model cache for critical commands
Data privacy breach	Critical - proprietary project data exposed	AES-256 encryption at rest, TLS 1.3 in transit, key rotation policy, optional on-premise model deployment
Configuration errors in GitHub repo	Medium - AI behavior breaks unexpectedly	Schema validation on push, dry-run mode on PRs, automatic rollback on health-check failure
Token cost overruns	Medium - budget exceeded unexpectedly	Per-project monthly budgets with hard caps, cost-aware routing (prefer cheaper models for routine tasks), real-time spend alerts

Risk	Impact	Mitigation Strategy
Low employee adoption	Low-Medium - AI features underutilized	Phased rollout with champion users, feedback loops, gamified engagement metrics, inline onboarding tutorials

Table 5. Key risks and corresponding mitigation strategies.